logo.png

RAG-based Grafana assistant

Mojca Kompara, Miha Meglič, and Andraž Sovinec

Abstract

The abstract goes here. The abstract goes here.

Keywords
Keyword1, Keyword2, Keyword3 ...

Advisors: Slavko Žitnik

Introduction

Grafana is a widely used open-source platform for data visualization, monitoring, and analysis. Because it offers many features and supports a wide range of use cases, its documentation is extensive. As a result, users may sometimes find it difficult to navigate efficiently. The goal of our project is to build an AI assistant that helps users access relevant parts of the documentation more easily. The assistant will retrieve and present information from the documentation based on the user's query. This could significantly reduce the time needed to solve specific problems, since users would no longer have to manually search through large sections of documentation. The target users are system administrators, developers, and casual Grafana users.

There are already existing solutions for this problem. Grafana includes a keyword-based search function, and it also provides its own AI assistant. However, that assistant is designed to provide more general information about Grafana, while our solution will focus specifically on the official documentation. Because of this narrower focus, our assistant should be able to deliver more precise and relevant answers to documentation-related questions.

The primary motivation for this project is to reduce the time it takes users to set up or learn something new when using grafana and provide them with instant, reliable answers. We are going to use the RAG approach. With this approach, the system first retrieves the most relevant sections of the documentation in response to a user query and then uses a language model to generate an answer based on those retrieved sources. This allows the assistant to provide responses that are more accurate, context-aware, and grounded in the documentation itself. As a result, the system can reduce irrelevant answers and help users find reliable information more quickly.

Related Work

In higher education, RAG-based chatbots have been successfully applied to student support. For instance, the College RAG Chatbot ingests college brochures, FAQs, and guides to answer admissions and academic queries accurately from verified sources [1]. Our AI assistant will not be used for student support, but this working chatbot could still give us a lot of helpful insight for making it.

To address complex queries over large document collections, Lee [2] proposes a RAG framework that decomposes multihop questions into single-hop subquestions using an LLM.

Proposed Data

For this project, we will use web-accessible documentation from the official Grafana webpage (https://grafana.com/docs/).

Initial implementation

We decided to use the Retrieval-Augmented Generation (RAG) approach to build our chatbot. This approach allows the model to generate answers based on relevant external documentation instead of relying only on its pre-trained knowledge. For our implementation, we used Ollama, an open-source tool that makes it easier to run large language models locally. To prepare the documentation for the RAG pipeline, we used the Python library LlamaIndex. This library helped us load the Grafana documentation files, process their contents, and split them into smaller chunks that could be stored and retrieved efficiently. After chunking the documentation, we embedded the chunks, stored them in a vector database, and retrieved the most relevant ones for each query before passing them to the language model.

Performance analysis

These results should be interpreted as an initial evaluation, since the benchmark contains only 20 queries and the quality scores are based on subjective human judgment.

The results varied in both quality and response time. The response times ranged from around 12 seconds up to 67 seconds. The quality of the responses was rated based on the subjective opinions of two separate evaluators. Most of the responses were good, but a few were very poor. Queries with shorter response times were usually those that required short and simple answers, while queries with longer response times were generally those that asked how to set up something and therefore required a description of a full process. The queries used for testing are listed below, and the metrics from the tests are shown in table 1.

Queries used for testing

1. ”What is Grafana used for?”
2. ”How do I create a new dashboard?”
3. ”Explain how to set up alerts.”
4. ”What data sources does Grafana support?”
5. ”How can I share a dashboard with others?”
6. ”Describe the process to install plugins.”
7. ”How do I manage user permissions?”
8. ”What is the default port for Grafana?”
9. ”How to upgrade Grafana to the latest version?”
10. ”Where are Grafana logs stored?”
11. ”How do I change the Grafana admin password?”
12. ”How do I add a Prometheus data source?”
13. ”Can Grafana send alert notifications?”
14. ”How do I back up Grafana?”
15. ”Where is the Grafana configuration file located?”
16. ”How do I invite another user to Grafana?”
17. ”How do I install a panel plugin?”
18. ”Can Grafana connect to Elasticsearch?”
19. ”How do I export a dashboard?”
20. ”What happens if Grafana cannot find a data source?”

Table 1. Model analysis

No. of question	Response time [s]	Rating
1	23.42	4/5
2	44.40	4/5
3	67.17	5/5
4	24.55	5/5
5	26.29	5/5
6	52.27	5/5
7	31.61	1/5
8	12.59	5/5
9	50.46	2/5
10	29.37	0/5
11	43.86	3/5
12	28.45	1/5
13	22.25	5/5
14	47.19	5/5
15	28.26	5/5
16	42.38	5/5
17	21.07	5/5
18	15.33	5/5
19	29.75	5/5
20	15.29	2/5
Average	32.80	3.85

Problematic cases

One case that we could consider problematic is the query “Explain how to set up alerts,” since it had the longest response time. It is true that this query requires a description of an entire process, but 67.17 seconds still feels too long. One possible reason for the long response time is that the model may have needed to combine information from multiple chunks in order to generate a response, instead of relying mainly on the best-matching chunk. The problem here is mainly the efficiency of collecting the information.

There are also some problematic cases where the model misses the main point of the question. Examples include the queries “Where are Grafana logs stored?” and “How do I manage user permissions?”. For the first query, the response described ways to store logs outside of Grafana instead of giving the location of Grafana logs. For the second query, the response described the different roles that can be assigned to users and the rules that should be followed, but it did not provide actual information on how to set those permissions. One possible explanation is that the retrieval step favored chunks containing matching keywords, even when those chunks did not answer the full intent of the question.

Future directions and ideas

One possible way to improve performance would be to clean the Grafana documentation by removing duplicates and organizing the source documents better. This can have a big effect on RAG systems, because cleaner and better-structured data usually leads to more accurate retrieval. We could also experiment with different chunking strategies to see whether any of them works significantly better than the others. For

example, some strategies may preserve context better, while others may improve retrieval precision.

Another way to improve performance is to improve retrieval. This could be achieved by using a hybrid retrieval approach, which combines lexical and vector methods. Such an approach could help the system retrieve both exact keyword matches and semantically similar content. We could also try adapting retrieval to be query-specific, or additionally reordering the chunks after the initial retrieval to improve the quality of the final context given to the model.

The final idea for improvement is to try out different models and compare their performance. Some models might be better suited to this specific task than others, especially when dealing with technical documentation such as Grafana docs. In addition to answer quality, we could also compare the models based on speed, consistency, and hardware requirements.

References

[1] Deep Shah. College rag chatbot. https://github.com/DeepShah1406/College_RAG_Chatbot, 2024. Accessed: March 20, 2026.

[2] Seokgi Lee. Transforming questions and documents for semantically aligned retrieval-augmented generation. *arXiv preprint arXiv:2508.09755*, 2025.